Laboratory
0b100

Expected submission ON PORTALE DELLA DIDATTICA:
SURNAME_NAME_MATRICOLA_lab_04.zip

Including (in case you have projects with the same name, you can prepend lab_4 to them):
- program_1
- program_1_r
- program_1_u
- lab_04.pdf (complete this file and export it as PDF)

Submission deadline: 27/10/2026

1) Create a new program (program_1) and copy into it the code provided below.

   The code performs one training step of a simplified AdaGrad optimizer, which gives every weight its own learning rate. For each of the N = 8 weights it accumulates the square of the gradient g[i] in h[i] and then updates the weight w[i] with a step that shrinks as h[i] grows (the real AdaGrad divides by √h[i] + ε; the square root is omitted here):

$$h[i] = h[i] + g[i]^2 \qquad (accumulated\ \ squared\ \ gradient)$$

$$w[i] = w[i] - lr \cdot \frac{g[i]}{h[i]} \qquad (weight\ \ update)$$

   program_1 is a straightforward translation of the C loop: it is correct, but the instructions are written in the order in which the formulas are read, and the same registers are reused for different values. In this laboratory you will make it faster without changing what it computes: only by re-scheduling instructions, renaming registers and unrolling the loop.
   Rules:
   - Run **program_1** and check that, at the end of the execution, h = [1.25, 3.0, 0.5625, 5.0, 4.25, 3.25, 0.3125, 3.0] and w ≈ [0.26, −0.6667, 1.1556, 0.01, −0.3882, 0.8538, −1.02, 0.5667]. Every program you write must produce exactly the same values.
   - Do not modify the .data section.
   - Pseudo-instructions (la, j, nop, …) are translated by the assembler into one or more real instructions: take this into account when you compute clock cycles and CPI by hand.

```
# program_1: one step of a (simplified) AdaGrad optimizer
        .data
# ---------- AdaGrad update of N = 8 weights ----------
N:      .word 8
lr:     .float 0.1                      # learning rate
g:        .float  0.5,  -1.0,  0.25,  2.0,  -0.5,  1.5,  -0.25,  1.0      #
gradients
h:      .float 1.0, 2.0, 0.5, 1.0, 4.0, 1.0, 0.25, 2.0       # accumulated
g^2
```

```
w:      .float 0.3, -0.7, 1.2, 0.05, -0.4, 0.9, -1.1, 0.6     # weights
.section .text
.globl _start
_start:
        la   t0, g                          # t0 -> g[i]
        la   t1, h                          # t1 -> h[i]
        la   t2, w                          # t2 -> w[i]
        la   t3, lr
        flw  f0, 0(t3)                      # f0 = lr
        la   t4, N
        lw   t5, 0(t4)                      # t5 = N (loop counter)
loop:   flw  f1, 0(t0)                      # f1 = g[i]
        flw  f2, 0(t1)                      # f2 = h[i]
        fmul.s f3, f1, f1                   # f3 = g[i]^2
        fadd.s f2, f2, f3                   # h[i] = h[i] + g[i]^2
        fsw  f2, 0(t1)
        fmul.s f3, f0, f1                   # f3 = lr * g[i]
        fdiv.s f3, f3, f2                   # f3 = lr * g[i] / h[i]
        flw  f4, 0(t2)                      # f4 = w[i]
         fsub.s f4, f4, f3                      # w[i] = w[i] - lr * g[i] /
h[i]
        fsw  f4, 0(t2)
        addi t0, t0, 4
        addi t1, t1, 4
        addi t2, t2, 4
        addi t5, t5, -1                     # one iteration less
        bne  t5, zero, loop
End:
        li a0, 0
        li a7, 93
        ecall
```

2) Use the following CPU configuration, run the program (**program_1**), observe the resulting pipeline and answer the following:



   a. Manually detect the different hazards that cause pipeline stalls. Report at least 4 hazards in the code (or the pipeline) and fill in the following table (**first line of the table is filled with a fake example, delete it**):

RISC-V Instruction(s)	Type of Hazard	Pipeline Stage	Cause of Stall
i.e. ADD x5, x1, x2 SLLI x6, x5, 1	i.e. Data - RAW	i.e. EX	i.e. x5 not ready for shift until ADD complete
…	…	…	…
Fmul.s f3, f1, f1 fadd.s f2, f2, f3	RAW		Fadd.s wants to read f3 but f3 is been processed by fmul.s because fmul.s has 6 clock cycles in its execution state so fadd.s need to stall until the value is available in the EX-

			MEM pipeline to be forwareded
Fadd.s f2,f2,f3 fsw f2, 0(t1)	RAW		Here fsw wants to store the value in f2 been executed by fadd.s so to overcome this fsw need to stall until the value is available
Fmul.s f3, f0, f1 fdiv f3, f3, f2	WAW		Here we can see that fmul wants to write in f3 and fdiv wants to do the same this is a WAW and thus an output dependency
Fmul.s f3, f0, f1 fdiv f3, f3, f2	RAW		We also have a situation here that f3 is been needed by fdiv but fmul is still processing it.

b. Consider the following instructions of program_1. Indicate which type of data dependence or dependences (RAW, WAR, WAW) exist between them and how each one is handled by the CPU. Motivate your answer.

```
(I1)  fmul.s f3, f1, f1
(I2)  fadd.s f2, f2, f3
(I3)  fmul.s f3, f0, f1
(I4)  fdiv.s f3, f3, f2
```

Answer:
I1-I2: Data dependency (RAW)

So here we can see that I1 and I2 uses a register f3 and I2 needs f3 that is the output of I1 but I1 is fmul so will take much clock cycles that I2 which is fadd and this will go out of order. To overcome this problem the cpu stall I2 until the value f3 is available between the pipeline Ex – MEM (if forwarding is accepted in this architecture).

I3-I4: Output Dependency(WAW) and Data Dependency(RAW)

Here we can see that I3 writes in f3 and also I4 writes in f3 so to resolve this the use of register renaming can be use.

Also f3 in been needed by fdiv and fmul compute the value so stalling the fdiv for a will is a possible solution by the cpu.

c. Considering program_1, indicate which types of hazards are present, which of them cause stalls in the pipeline and how they are handled by the CPU. Motivate your answer.

Answer:
RAW hazard:

Fmul.s f3, f1, f1
fadd.s f2, f2, f3

we can see that here fadd reads f3 that is not yet written by fmul and fmul take more time to complete compered to fadd.

WAW harzard:

Fmul.s f3, f0, f1

fdiv f3, f3, f2

we can se that mul writes to f3 and also fdiv does the same that is output dependence

3) Optimize program_1 as follows:
   a. **Re-schedule**. Optimize program_1 by re-scheduling instructions and renaming registers to eliminate as many hazards as possible. You may change the order of the instructions, the registers they use and the offsets of loads and stores, but no instruction can be added or removed. Manually calculate the number of clock cycles for the new program (program_1_r) to execute and compare the results with those obtained by the simulator.
   b. **Unroll**. Unroll the loop of program_1 with an unrolling factor of 2, then re-schedule the instructions and rename the registers. Manually calculate the number of clock cycles to execute the new program (program_1_u) and compare the results obtained with those obtained by the simulator.
   c. **Reschedule, unroll and rename**. Apply the following optimizations to program_1: (i) reschedule instructions (ii) unroll the loop with an unrolling factor of 4 (iii) reschedule instructions in the unrolled loop and use register renaming to further optimize the program. Manually calculate the number of clock cycles for the new program (program_1_o) and compare the results obtained with those obtained by the simulator.

   Complete the following table with the results obtained:

Program	program_1	program_1_r	program_1_u	program_1_o
Clock cycles by hand	487	487	512	
Clock cycles by simulation	479.16	431.64	414.8	

4) Considering program_1, would it be more convenient to reduce by 2 clock cycles the latency of the FP multiplier (6 → 4) or the latency of the FP divider (30 → 28)? Answer first by reasoning on the pipeline, then verify your answer with the simulator by changing the CPU configuration. Repeat the analysis for program_1_r, program_1_u and program_1_o. Does the answer change? Motivate your answer.

Clock cycles (simulation)	program_1	program_1_r	program_1_u	program_1_o
Initial configuration	479.16	431.64	414.8	
FMUL: 4 cycles	447.48	414.48	398.48	
FDIV: 28 cycles	463.32	414.48	398.48	

Answer:

reducing the latency of the FDIV form 30 to 28 improve the CPI to and the clock per cycle from 479.16 to 463.32 because fdiv makes stalling more instructions that fmul and for a considerable clock

5) Collect the Cycles Per Instruction (CPI) from the simulator for different programs with different configurations (initial, FMUL: 4 cycles, FDIV: 28 cycles)

Initial configuration	program_1	program_1_r	program_1_u	program_1_o
CPI by hand	3.74	3.74	3.82	
CPI by simulation	3.27	3.02	2.93	

FMUL: 4 cycles	program_1	program_1_r	program_1_u	program_1_o
CPI by hand	3.52	3.52	3.44	
CPI by simulation	3.39	3.14	2.93	

FDIV: 28 cycles	program_1	program_1_r	program_1_u	program_1_o
CPI by hand	3.64	3.64	3.49	
CPI by simulation	3.51	3.14	2.93	

Compare the results obtained and provide some explanation if the results are different.

Explanation (if any):

6) Calculate by hand the clock cycles and Cycles Per Instruction (CPI) for program_1 and program_1_o assuming that the branch delay slot is enabled

Initial configuration	program_1	program_1_o	program_1 (branch delay slot)	program_1_o (branch_delay_slot)
CPI by hand				
Clock cycles by hand				

Compare the results obtained and provide some explanation if the results are different

Explanation (if any):